Cache Memory

Lecture Notes — Week 9

PCC CS-401: Computer Organization and Architecture

August 18, 2026

9.1 Introduction: Why Main Memory Needs a Cache

Week 8 ended by asking a question it could not itself answer. Polling, interrupts, DMA, and I/O processors were four different answers to one question — a device is far slower than the CPU, so *who does the waiting, and what does the waiting cost?* — and every answer moved that cost somewhere else, but none made it vanish. Week 8 left one gap explicitly open: once the data has actually been delivered into main memory, the CPU *still* waits for it. DRAM is roughly a hundred times slower than the processor that reads it. The identical speed-mismatch problem that Week 8 solved for I/O devices now reappears *inside* the machine, between the CPU and its own main memory. This chapter is that problem's answer: the memory hierarchy in general, and the cache in particular.

The magnitude of the mismatch is worth putting a number on before any machinery is introduced. A 2 GHz CPU issues one instruction every 0.5 ns. A DRAM access takes about 60 ns. If *every* instruction needed one memory access, the CPU would spend $60 \div 0.5 = 120$ instruction slots waiting for memory for every single access — it would run at roughly 1/120 of its rated speed. We cannot make DRAM 120× faster at DRAM prices, so the question has to change. Instead of asking "can we make *all* of memory fast?", we ask "can we make the *common case* fast?" That is exactly what the memory hierarchy does, and the rest of this chapter is organized around six questions that build up to that answer:

1. **The memory hierarchy** (Section 9.2) — why fast, big, and cheap cannot be bought together, and how *locality* lets us fake it.
2. **Main memory** (Section 9.3) — SRAM, DRAM, and the ROM family.
3. **Cache principles** (Section 9.4) — blocks, lines, tags, hits, and misses.
4. **Mapping functions** (Section 9.5) — direct, fully associative, and k-way set associative, derived field-by-field from the address.
5. **Replacement algorithms** (Section 9.6) — LRU, FIFO, LFU, Random, and Optimal, all traced on one common reference string.
6. **Performance** (Section 9.7) — hit ratio, AMAT, and multi-level effective access time.

Two running threads carry the whole chapter. The first is one *system* — 64 KB main memory, 2 KB cache, 32-byte blocks — and one address, `0x3F8C`, carried through all three mapping schemes so the schemes can be compared on identical inputs. The second is one 13-reference string carried through all five replacement policies, so the policies can be compared on identical inputs too.

9.2 The Memory Hierarchy

The memory hierarchy exists because of a constraint the designer cannot escape. Anchor reading for this section is Stallings, *Computer Organization and Architecture*, 11th ed., Ch. 4 (printed pp. 112–137).

9.2.1 Three Characteristics That Refuse to Cooperate

Across every memory technology, three characteristics move together in a way we cannot break [2, Ch. 4, printed pp. 112–137]:

- Faster access time ⇒ greater cost per bit.
- Greater capacity ⇒ smaller cost per bit.
- Greater capacity ⇒ slower access time.

Read the three together and the trap is clear: the memory we want — fast *and* big *and* cheap — cannot be built from any single technology, because each characteristic fights the other two. The designer cannot beat this trade-off; the designer can only arrange several technologies so that the fast one is used *most of the time*. Before that arrangement is possible, a few terms need to be pinned down. **Access time** (latency) is the time from "read this address" to "here is the data." **Memory cycle time** is the access time plus any recovery the technology needs before the next access. And the **access method** names how a memory is reached: *sequential* (tape — position matters), *direct* (disk — seek then read), *random* (RAM — every location costs the same time), and *associative* (cache tags — compared by *content*, not address) [2, Table 4.1, PDF p. 156].

9.2.2 Locality of Reference: Why the Trick Works

If memory references were spread uniformly across all of memory, a small fast level would be useless — it would be right only a tiny fraction of the time. The entire hierarchy stands on the observation that real program references are *not* uniform. They cluster.

Definition (Locality of Reference). *Memory references made by a running program are not spread uniformly; they cluster [2, §4.1, printed pp. 112–117].* ***Temporal locality*** *is the tendency for a location referenced now to be referenced again soon (a loop counter, the top of the stack).* ***Spatial locality*** *is the tendency for locations* near *one just referenced to be referenced soon (array traversal, sequential instruction fetch).*

Each form of locality pays for one specific design decision. Spatial locality is why, on a miss, we move a whole *block* of neighbouring words into the cache rather than just the single word that was asked for — the neighbours are about to be needed anyway. Temporal locality is why keeping a block *around* after it is loaded pays off — and, anticipating Section 9.6, why a replacement policy that throws away recently-used data is a bad one.

9.2.3 The Pyramid: Fast at the Top, Big at the Bottom

Figure 9.1 draws the hierarchy as a five-tier pyramid, wide at the bottom and narrow at the top, so that the three non-cooperating characteristics of this section read off directly from the shape. Each level is a cache for the level below it: registers hold what the current *instruction* needs; cache holds what the current *loop* needs; main memory holds what the current *program* needs [2, Fig. 4.6, PDF p. 160].

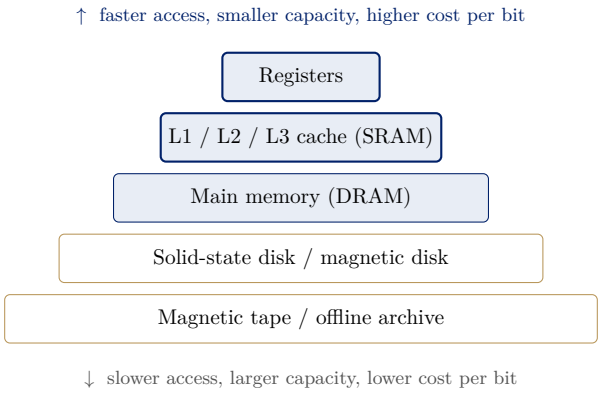

Figure 9.1: The five-tier memory hierarchy: fast and small and expensive at the top, slow and large and cheap at the bottom. Each level caches the level below it — cf. Stallings, *Computer Organization and Architecture*, 11th ed., Fig. 4.6, PDF p. 160 [2].

This chapter studies exactly one boundary of the pyramid — the one between cache and main memory — because that is where the CPU's speed mismatch is sharpest, and where the mapping and replacement machinery of Sections 9.5 and 9.6 is designed.

9.2.4 The Two-Level Performance Model

To quantify what the hierarchy buys, model the boundary between two adjacent levels. Let level 1 (the cache) have access time T_1 and **hit ratio** H — the fraction of references found in level 1 — and let level 2 (main memory) have access time T_2. A miss must *first* search level 1 and only then go to level 2, so a hit costs T_1 while a miss costs $T_1 + T_2$. Weighting by the hit and miss probabilities:

$$T_{\text{avg}} \;=\; H\,T_1 \;+\; (1-H)\,(T_1+T_2) \;=\; T_1 + (1-H)\,T_2.$$

The whole hierarchy stands or falls on H being close to 1 — and locality is what makes that possible [2, §4.3, Fig. 4.8, PDF p. 162]. It is important to be clear about what H *is* and *is not*: H is not a design knob you set. It is an *outcome* of cache size, block size, mapping, and replacement policy — the rest of this lecture.

Example 9.2.1 (What a Hit Ratio Buys)**.** *Take a cache with access time $T_1 = 1$ ns and main memory with access time $T_2 = 100$ ns. Compute T_{avg} at $H = 0.50$, $H = 0.90$, and $H = 0.95$, using $T_{avg} = 1 + (1 - H) \times 100$:*

- *$H = 0.50$: $T_{avg} = 1 + 0.50 \times 100 = 51$ ns — barely better than no cache at all.*
- *$H = 0.90$: $T_{avg} = 1 + 0.10 \times 100 = 11$ ns.*
- *$H = 0.95$: $T_{avg} = 1 + 0.05 \times 100 = 6$ ns — a 16.8× speed-up over raw DRAM.*

Notice the shape of the payoff. Going from $H = 0.5$ to $H = 0.9$ saves $51 - 11 = 40$ ns; going from $H = 0.9$ to $H = 0.95$ saves only $11 - 6 = 5$ ns more. The payoff is violently non-linear, which is why real designs fight for the last percentage point of hit ratio.

9.3 Main Memory: SRAM, DRAM, and the ROM Family

Before the cache can stand in for main memory, it is worth one short detour to ask what main memory is actually made of — and to see that the SRAM/DRAM split is itself the hierarchy in miniature. Anchor reading for this section is Stallings 11e, Ch. 6 §6.1 (PDF pp. 226–236).

9.3.1 Two Kinds of Read/Write Memory

Definition (SRAM and DRAM). *Both SRAM and DRAM are* volatile *(they lose their contents when power is removed) and both are "random access" (every address costs the same time).* ***SRAM*** *(static RAM) stores each bit in a flip-flop built from four to six transistors: no refresh is needed, and it is fast, expensive, and low density — this is what* cache *is built from.* ***DRAM*** *(dynamic RAM) stores each bit as charge on a tiny capacitor, one transistor per cell: it is dense and cheap, but the charge leaks, so every cell must be periodically* refreshed *— this is what* main memory *is built from [2, §6.1, PDF pp. 226–236].*

Two clarifications ward off common confusions. "Random access" means every address costs the same time; it does *not* mean the contents are random. And the SRAM/DRAM split is exactly the hierarchy of Section 9.2 in miniature — the same speed-versus-density trade-off, resolved twice inside the same machine.

9.3.2 The ROM Family: Non-Volatile "Read-Mostly" Memory

The other family of semiconductor memory is the read-only (really, *read-mostly*) memory used for boot firmware, microcontroller program store, and lookup tables. Table 1 runs the family from cheapest-but-unchangeable to most-flexible-but-costliest [2, §6.1, PDF pp. 226–236].

Type	How it is written	How it is erased	Volatile?
ROM (mask)	At fabrication, by masking	Not possible	No
PROM	Electrically, once, by the user	Not possible	No
EPROM	Electrically	UV light, whole chip	No
EEPROM	Electrically, byte at a time	Electrically, byte-level	No
Flash	Electrically	Electrically, block-level	No

Table 1: The ROM family, from mask ROM (written once at fabrication, unchangeable, only economic in high volume) through PROM, EPROM, EEPROM, to flash [2, §6.1, PDF pp. 226–236].

Flash sits deliberately in between: it is erasable in *blocks*, not bytes — cheaper and denser than EEPROM, more flexible than EPROM. That is why it is the technology behind SSDs.

9.4 Cache Principles

With the hierarchy motivated and main memory characterized, the main event is the cache itself: the small, fast SRAM placed between the CPU and DRAM. Anchor reading for this section is Stallings 11e, Ch. 5 §5.1 (printed pp. 139–142).

9.4.1 Definition: Cache, Block, Line, Tag

Definition (Cache Memory). *A* ***cache*** *is a small, fast SRAM memory placed between the CPU and main memory. Main memory is divided into fixed-size* ***blocks*** *of 2^b bytes; the cache holds a*

*small number of **lines**, each able to store* one *block plus a **tag** identifying* which *block it currently holds [2, §5.1, printed pp. 139–142].*

Because there are far more main-memory blocks than cache lines, the tag is unavoidable: a line cannot be self-identifying, since the same line must be able to hold any of many blocks at different times. Each line also carries a **valid bit** (has this line ever been loaded?), and, under write-back (Section 9.7.3), a **dirty bit** (has it been modified since loading?). A **hit** means the requested block is present; a **miss** means it is not, so the block is fetched from main memory and installed into a line.

9.4.2 How the CPU Reads Through a Cache

The read path is a four-step sequence [2, Fig. 5.3, PDF p. 182]:

1. The CPU issues an address.
2. The cache splits that address into fields (Section 9.5) and checks whether the requested block is resident, by comparing tags.
3. **Hit:** the required word is selected from the line using the *word/offset* field and delivered to the CPU. One cache access, done.
4. **Miss:** the cache requests the whole block from main memory, installs it in a line (possibly evicting an existing block — Section 9.6), and then delivers the word.

The one thing to hold on to from this path: a miss costs the *whole block transfer*, not one word. That is a bet on spatial locality — we pay for 2^b bytes hoping the neighbours get used too.

9.4.3 The Seven Elements of Cache Design

The cache designer turns seven knobs, and this taxonomy is the spine of the whole chapter [2, Table 5.1, PDF p. 184]:

- **Cache addresses** — logical (virtual) or physical.
- **Cache size** — small enough to be fast, big enough to hold the working set.
- **Mapping function** — direct, associative, set associative (Section 9.5).
- **Replacement algorithm** — LRU, FIFO, LFU, Random (Section 9.6).
- **Write policy** — write through, write back (Section 9.7.3).
- **Line size** — how many bytes per block.
- **Number of caches** — single vs. multi-level, unified vs. split.

The two bolded rows — mapping function and replacement algorithm — are the two the rest of this chapter is about.

9.5 Mapping Functions

There are far more main-memory blocks than cache lines, so when block number B arrives, the hardware must answer two questions *fast*: (1) **placement** — which line(s) is B *allowed* to occupy? — and (2) **identification** — given a candidate line, is the block sitting in it actually B? The three mapping schemes are three different answers to these two questions, arranged along a single axis of "how much placement freedom, at what identification cost." Anchor reading for this section is Stallings 11e, Ch. 5 §5.2 (PDF pp. 187–203).

9.5.1 The Master Framework: Splitting an Address

All three schemes are one formula. Let main memory hold 2^N addressable bytes (so the physical address is N bits), let the block size be 2^b bytes, and let the cache hold $L =$ cache size/block size lines. Every address splits into three fields:

$$\underbrace{\text{tag}}_{\text{identify}} \quad \underbrace{\text{index}}_{\text{place}} \quad \underbrace{\text{word / offset } (b \text{ bits})}_{\text{select byte in block}}$$

with the field widths determined by the scheme [2, §5.2, PDF pp. 187–203]:

- **Direct:** index $= \log_2 L$ line bits; tag $= N - \log_2 L - b$.
- **Fully associative:** no index at all; tag $= N - b$.
- **k-way set associative:** $S = L/k$ sets, index $= \log_2 S$ set bits; tag $= N - \log_2 S - b$.

The three schemes are one formula with the index field shrinking from $\log_2 L$ bits, through $\log_2(L/k)$, to zero — and every bit the index loses, the tag gains. The whole section follows from this observation, so it is worth stating plainly before the worked examples: *derive the fields; never memorise them.*

Example 9.5.1 (The Running System for All Mapping Examples)**.** *System A, fixed for every worked example that follows: main memory* $= 64$ *KB* $= 2^{16}$ *bytes, so the physical address is* $N = 16$ *bits; cache* $= 2$ *KB* $= 2^{11}$ *bytes; block (line) size* $= 32$ *bytes* $= 2^5$*, so* $b = 5$*. Two derived quantities are used constantly:*

- *Number of cache lines* $L = 2^{11}/2^5 = 2^6 = 64$ *lines.*
- *Number of main-memory blocks* $= 2^{16}/2^5 = 2^{11} = 2048$ *blocks.*

So 2048 blocks compete for 64 lines — 32 blocks per line — and that ratio is the whole reason mapping and replacement exist. The probe address throughout is `0x3F8C` $= 16268_{10}$*, whose block number is* $\lfloor 16268/32 \rfloor = 508$ *with byte offset* $16268 - 508 \times 32 = 12$*.*

Figure 9.2 shows the same 16-bit address split three ways, with field widths drawn proportional to bit count so the word-field boundary aligns across all three rows — that alignment is the teaching point.

What Figure 9.2 shows: the word field never moves — it is fixed by the block size alone. Everything to its left is a fixed budget of 11 bits that the index and tag *share*. Less freedom in placement means a narrower tag; more freedom means a wider one.

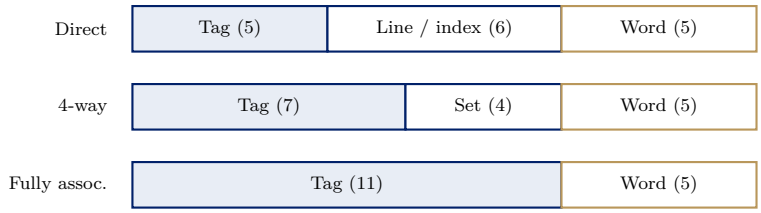

Figure 9.2: System A's 16-bit address split three ways: direct (5 | 6 | 5), 4-way set associative (7 | 4 | 5), and fully associative (11 | 5). The word field never moves; the 11 bits to its left are a fixed budget that the index and tag share.

9.5.2 Direct Mapping

Definition (Direct Mapping). *Block B of main memory may occupy* exactly one *cache line, given by*

$$line \;=\; B \bmod L, \qquad tag \;=\; \lfloor B/L \rfloor.$$

The line number is read straight out of the address — no search, no choice [2, Fig. 5.7, PDF p. 190].

Blocks B, $B+L$, $B+2L$, … all land on the same line, and are told apart *only* by the tag. This is the cheapest possible hardware — one line decoder and one tag comparator — but the cost is the mirror image of the saving: two hot blocks that happen to be L apart evict each other forever, no matter how empty the rest of the cache is.

Figure 9.3 makes the many-to-one mapping visible with a toy cache of $L = 4$ lines so the modulo can be read directly.

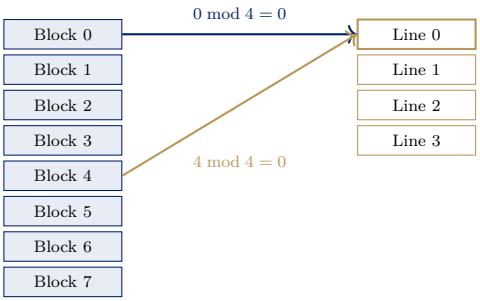

Figure 9.3: Direct mapping on a toy cache of $L = 4$ lines: blocks 0, 4, 8, 12, … are all forced onto line 0 (and 1, 5, 9, … onto line 1, etc.). Only one may be resident at a time, and the tag records which.

Figure 9.4 shows the hardware behind the lookup: the line field is a *decoder input*, not something to search for. Only after the line is selected does a single comparison decide hit or miss.

Example 9.5.2 (Direct Mapping, Worked: Deriving the Bit Widths)**.** *For System A (64 KB memory, 2 KB cache, 32-byte blocks), derive the width of every address field, step by step:*

1. *Physical address width:* $64\ KB = 2^{16}\ bytes \Rightarrow N = 16\ bits.$
2. *Word (offset) field:* $block = 32 = 2^5\ bytes \Rightarrow b = 5\ bits.$
3. *Number of lines:* $L = 2\ KB/32\ B = 2^{11}/2^5 = 2^6 = 64 \Rightarrow$ *line field* $= \log_2 64 = 6\ bits.$
4. *Tag field* $= N - \log_2 L - b = 16 - 6 - 5 = 5\ bits.$

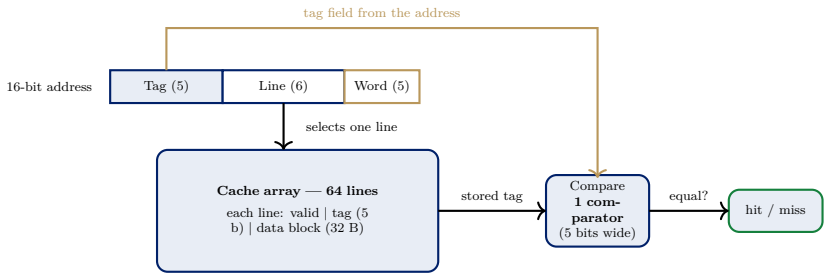

Figure 9.4: Direct-mapped lookup datapath: the line field selects one line, the array emits that line's stored tag, and a single comparator checks it against the address's tag field.

Check: $5+6+5=16$. *The three fields must always re-sum to the full address width; if they do not, one of the four steps above is wrong.*

Example 9.5.3 (Direct Mapping, Worked: Tracing Address `0x3F8C`)**.** *Which line does* `0x3F8C` *map to, and what is its tag? Write* `0x3F8C` $= 0011\,1111\,1000\,1100_2$ *and slice it* $5 \mid 6 \mid 5$ *from the left:*

Field	***Bits***	***Value***
Tag (5 b)	`00111`	7
Line (6 b)	`111100`	60
Word (5 b)	`01100`	12

Now verify by arithmetic, not by bit-slicing. `0x3F8C` $= 16268$*; block* $B = \lfloor 16268/32 \rfloor = 508$*; offset* $= 16268 - 508 \times 32 = 12$*. Then line* $= B \bmod L = 508 \bmod 64 = 508 - 448 = 60$*, and tag* $= \lfloor B/L \rfloor = \lfloor 508/64 \rfloor = 7$*. Two independent routes give the same answer; in an exam, do one and check with the other.*

Example 9.5.4 (Direct Mapping's Weakness: Conflict Misses)**.** *A loop alternately touches address* `0x0780` *(block 60) and* `0x0F80` *(block 124), both in System A. What is the hit ratio?*

$60 \bmod 64 = 60$ *and* $124 \bmod 64 = 60$ — *both map to line 60, with tags* $\lfloor 60/64 \rfloor = 0$ *and* $\lfloor 124/64 \rfloor = 1$*. Every reference finds the* other *block's tag in line 60, so every reference is a miss: hit ratio* $= 0\%$*, while 63 of the 64 lines sit empty. This is a* ***conflict (collision) miss*** *— not a capacity problem, purely a placement-restriction problem.*

The fix previews set associativity: in a 2-way set-associative cache ($S = 64/2 = 32$ *sets),* $60 \bmod 32 = 28$ *and* $124 \bmod 32 = 28$ — *still the same set, but now the set has* two *ways, so both blocks stay resident and the hit ratio jumps to nearly* 100%*.*

9.5.3 Fully Associative Mapping

Definition (Fully Associative Mapping). *A block may be placed in* any *cache line. There is no index field: the address splits into tag | word only, and the tag is simply the block number [2, Fig. 5.10, PDF p. 195]:*

$$tag = B = \lfloor address/2^b \rfloor, \qquad tag\ width = N - b.$$

The two consequences are exact opposites. On the good side, there are *no conflict misses ever*: a block is evicted only when the cache is genuinely full, so the only misses left are compulsory

and capacity misses. On the bad side, identification is now a *search*: with no index to narrow the candidates, *every* line's tag must be compared, simultaneously. That is what **content-addressable memory (CAM)** does — a memory searched by content rather than by address [2, PDF pp. 193–194].

Example 9.5.5 (Fully Associative, Worked: `0x3F8C` Again)**.** *Re-derive from scratch for System A. Only the block size changes the word field; everything else the tag absorbs.*

1. *Word field* $= b = 5$ *bits (unchanged — 32-byte block).*
2. *There is no index field, so tag* $= N - b = 16 - 5 = 11$ *bits.*
3. *Check:* $11 + 5 = 16$.
4. *Slice* `0x3F8C` $= 0011\,1111\,1000\,1100_2$ *as* $11 \mid 5$*: tag* $=$ `00111111100`$_2 = 508$*, word* $=$ `01100`$_2 = 12$.
5. *Verify: the tag* is *the block number, and Example 9.5.3 computed* $B = 508$.

The answer to "which line?" is any of the 64 *— that is precisely what fully associative means, and precisely why a replacement policy becomes necessary.*

9.5.4 The Price of Full Associativity

Count the hardware for System A. Fully associative needs 64 comparators, one per line, each 11 bits wide, all firing in parallel on every access, plus $64 \times 11 = 704$ tag bits. Direct mapping needs 1 comparator of 5 bits and $64 \times 5 = 320$ tag bits. So full associativity costs 2.2× the tag storage and 64× the comparators — and the comparator array sits on the critical path, so it also costs *access time*. Real caches with hundreds or thousands of lines cannot afford this; hence the compromise that follows.

9.5.5 k-Way Set-Associative Mapping

Definition (k-Way Set-Associative Mapping). *Group the* L *lines into* $S = L/k$ ***sets*** *of* k *lines each. A block is mapped to* one *set by index, and may occupy* any *of the* k *lines ("ways") inside it [2, Fig. 5.13, PDF p. 200]:*

$$set = B \bmod S, \qquad tag = \lfloor B/S \rfloor, \qquad tag\ width = N - \log_2 S - b.$$

Two limit cases show that set associativity is not a third idea but the *dial* between the other two: $k = 1$ gives $S = L$, which is exactly direct mapping; $k = L$ gives $S = 1$, which is exactly fully associative mapping. Real designs use the settings in between, typically $k = 2, 4, 8$.

Figure 9.5 shows the 4-way lookup hardware. It is deliberately the same skeleton as Figure 9.4 — only the comparator count and the width of the index field change, which is the pedagogical point.

Example 9.5.6 (4-Way Set Associative, Worked: Bit Widths)**.** *System A again, now organised 4-way* ($k = 4$)*:*

1. *Address width* $N = 16$*; word field* $b = 5$ *(both unchanged — neither depends on associativity).*
2. *Lines* $L = 64$ *(unchanged — cache size and block size alone fix it).*

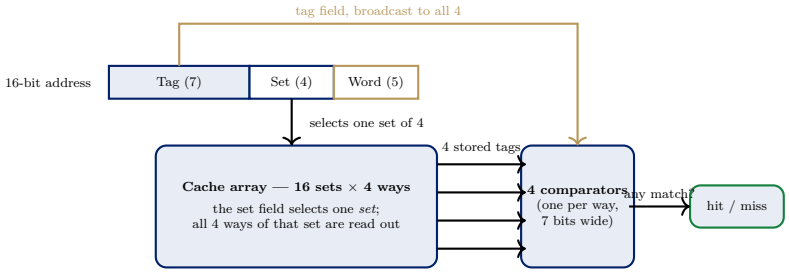

Figure 9.5: 4-way set-associative lookup datapath: the set field selects one set of four ways, all four stored tags are read out, and four comparators check each against the broadcast tag field. Identical skeleton to Figure 9.4; one comparator becomes k, and two bits move from the index field into the tag.

3. *Number of sets* $S = L/k = 64/4 = 16 \Rightarrow$ *set field* $= \log_2 16 = 4$ *bits.*
4. *Tag* $= N - \log_2 S - b = 16 - 4 - 5 = 7$ *bits.*
5. *Check:* $7 + 4 + 5 = 16$.

Notice what moved: going from direct (k=1*) to 4-way, the index lost 2 bits (*$6 \to 4$*) and the tag gained exactly those 2 (*$5 \to 7$*). Doubling* k *always trades one index bit for one tag bit.*

Example 9.5.7 (4-Way Set Associative, Worked: Tracing `0x3F8C`)**.** *Which* set *does* `0x3F8C` *map to, and what is its tag? Slice* `0x3F8C` $= 0011\,1111\,1000\,1100_2$ *as* $7 \mid 4 \mid 5$*:*

Field	***Bits***	***Value***
Tag (7 b)	`0011111`	31
Set (4 b)	`1100`	12
Word (5 b)	`01100`	12

Arithmetic cross-check (block $B = 508$*, as before): set* $= B \bmod S = 508 \bmod 16 = 508 - 496 = 12$*; tag* $= \lfloor B/S \rfloor = \lfloor 508/16 \rfloor = 31$*; word offset* $= 12$*, unchanged in all three schemes. The answer to "which line?" is now* any of the 4 ways of set 12 *— which is exactly where a replacement policy is finally needed.*

9.5.6 Hardware Cost, Counted

Table 2 collects the hardware cost across associativities for System A. All rows are the same 64 lines and the same 32-byte blocks; only the *grouping* changes.

Organisation	**Sets** S	**Index bits**	**Tag bits**	**Comparators**	**Total tag storage**
Direct (k=1)	64	6	5	1	$64 \times 5 = 320$ b
2-way	32	5	6	2	$64 \times 6 = 384$ b
4-way	16	4	7	4	$64 \times 7 = 448$ b
8-way	8	3	8	8	$64 \times 8 = 512$ b
Fully associative	1	0	11	64	$64 \times 11 = 704$ b

Table 2: Hardware cost versus associativity for System A (64 lines, 32-byte blocks throughout). Comparator count doubles with every doubling of k; tag storage grows only one bit per line.

The comparators, not the tag bits, are what make full associativity unaffordable: comparator count *doubles* with every doubling of k, while tag storage grows only *one bit per line*. (One valid bit per line throughout — and one dirty bit under write-back — is constant across all rows, so it does not change the comparison.)

9.5.7 The Three Schemes, Head to Head

Table 3 summarises the whole section. The unifying observation is stated after the table: **one dial, two end-stops** — $k = 1$ and $k = L$ are the extremes of the same formula, and every real design lives somewhere in between [2, §5.2, PDF pp. 187–203].

	Direct	**k-way set associative**	**Fully associative**
Address split	tag | line | word	tag | set | word	tag | word
Index width	$\log_2 L$	$\log_2(L/k)$	0
Tag width	$N - \log_2 L - b$	$N - \log_2(L/k) - b$	$N - b$
Placement rule	line $= B \bmod L$	set $= B \bmod (L/k)$	anywhere
Candidate lines	1	k	all L
Comparators	1	k	L
Replacement policy	none needed	needed, within the set	needed, over the whole cache
Conflict misses	severe	reduced, roughly as k grows	none
Search hardware	decoder only	decoder + k comparators	full CAM
Access time	fastest	intermediate	slowest
Typical use	simple / large L2–L3	L1 and L2 in practice	TLBs, small victim caches

Table 3: The three mapping schemes head to head, in terms of the one formula of Section 9.5.

Example 9.5.8 (Choosing an Organisation)**.** *An L1 data cache must answer in one CPU cycle; a last-level L3 cache may take twenty cycles but must not waste its (large) capacity. Which associativity for each, and why?*

- ***L1:*** *low associativity (2- or 4-way). The comparator array is on the critical path, and a 1-cycle budget cannot afford 64 parallel comparisons. Some conflict misses are the accepted price.*
- ***L3:*** *higher associativity (8- or 16-way). Latency is already large, so a wider comparator array is affordable — and with a big cache, wasting capacity to conflicts is the more expensive mistake.*

The general rule: associativity buys hit ratio and costs access time. Spend it where access time is not the binding constraint.

9.6 Replacement Algorithms

A replacement policy is needed only where placement offers a choice. Direct mapping needs none at all — a block has exactly one legal line, and if that line is occupied, the occupant is evicted, with no decision to make and no policy to implement [2, §5.2 Replacement Algorithms, PDF p. 203]. Fully associative mapping chooses the victim among *all* L lines; k-way set associative chooses among the k *lines of the target set*, never across sets. One further constraint matters enormously: to be useful at cache speed, a policy must be implementable *entirely in hardware* — no software bookkeeping, no sorting — which eliminates many policies that look good on paper.

9.6.1 Five Policies

Definition (LRU (Least Recently Used)). *Evict the line unreferenced for longest. Exploits temporal locality directly, and is the most popular policy in practice [2, §5.2, PDF p. 203].*

Definition (FIFO (First In, First Out)). *Evict the line resident longest, regardless of use. Implementable as a round-robin/circular buffer.*

Definition (LFU (Least Frequently Used)). *Evict the line with the smallest reference count.*

Definition (Random). *Evict a line chosen at random. Simulation studies report only slightly inferior performance to the usage-based algorithms [2, §5.2, PDF p. 203].*

Definition (OPT (Optimal / Belady's MIN) — standard general treatment, no page cite). *Evict the line whose* next *reference is furthest in the future. OPT is not in Stallings at all; it is presented as standard general treatment with no page citation, and it is* unimplementable *— it requires knowing the future. Its role is as a* benchmark*: no real policy can beat it, so it bounds how much a better policy could possibly gain.*

9.6.2 The Common Reference String

To compare the policies fairly, they are all run on one experiment: a *fully associative* cache with 3 lines, initially empty, receiving the following stream of *block numbers*:

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2 \qquad \text{(13 references)}$$

Three lines makes every eviction a genuine three-way choice, so the policies actually diverge — which is the point. In the traces that follow, each column is one reference, the three rows are the three cache lines (slots), `M` marks a miss and `H` a hit. The first three references are *compulsory misses*: an empty cache must miss on first touch, whatever the policy, so every policy scores at least 3 misses, and only the remaining 10 references separate them.

Example 9.6.1 (Trace 1 — FIFO)**.** *FIFO takes victims in strict round-robin order — slot 0, 1, 2, 0, … — never looking at whether a block is being used:*

Reference	*7*	*0*	*1*	*2*	*0*	*3*	*0*	*4*	*2*	*3*	*0*	*3*	*2*
Slot 0	*7*	*7*	*7*	***2***	*2*	*2*	*2*	***4***	*4*	*4*	***0***	*0*	*0*
Slot 1	*–*	*0*	*0*	*0*	*0*	***3***	*3*	*3*	***2***	*2*	*2*	*2*	*2*
Slot 2	*–*	*–*	*1*	*1*	*1*	*1*	***0***	*0*	*0*	***3***	*3*	*3*	*3*
Hit/Miss	*M*	*M*	*M*	*M*	*H*	*M*	*M*	*M*	*M*	*M*	*M*	*H*	*H*

Reference 7 is the tell: block 0 was referenced only two steps earlier, yet FIFO evicts it because it happened to arrive early. ***Result: 3 hits, 10 misses*** *— hit ratio* $3/13 = 23.1\%$.

Example 9.6.2 (Trace 2 — LRU)**.** *LRU evicts the line unreferenced for the longest time:*

Reference	*7*	*0*	*1*	*2*	*0*	*3*	*0*	*4*	*2*	*3*	*0*	*3*	*2*
Slot 0	*7*	*7*	*7*	***2***	*2*	*2*	*2*	***4***	*4*	*4*	***0***	*0*	*0*
Slot 1	*–*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	***3***	*3*	*3*	*3*
Slot 2	*–*	*–*	*1*	*1*	*1*	***3***	*3*	*3*	***2***	*2*	*2*	*2*	*2*
Hit/Miss	*M*	*M*	*M*	*M*	*H*	*M*	*H*	*M*	*M*	*M*	*M*	*H*	*H*

At reference 6, the resident blocks are $\{2, 0, 1\}$ *with block 1 unreferenced since step 3 — so LRU evicts* block 1*, where FIFO evicted block 0. That single better decision buys the extra hit at reference 7.* ***Result: 4 hits, 9 misses*** *— hit ratio* $4/13 = 30.8\%$*, beating FIFO.*

Example 9.6.3 (Trace 3 — LFU)**.** *LFU evicts the line with the smallest reference count, breaking ties by age (oldest first):*

Reference	*7*	*0*	*1*	*2*	*0*	*3*	*0*	*4*	*2*	*3*	*0*	*3*	*2*
Slot 0	*7*	*7*	*7*	***2***	*2*	*2*	*2*	***4***	*4*	***3***	*3*	*3*	*3*
Slot 1	–	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*0*
Slot 2	–	–	*1*	*1*	*1*	***3***	*3*	*3*	***2***	*2*	*2*	*2*	*2*
Count of 0	–	*1*	*1*	*1*	*2*	*2*	*3*	*3*	*3*	*3*	*4*	*4*	*4*
Hit/Miss	*M*	*M*	*M*	*M*	*H*	*M*	*H*	*M*	*M*	*M*	*H*	*H*	*H*

Block 0's count climbs to 3 by reference 7, so LFU protects it *at reference 8 — where LRU evicted it at reference 10.* ***Result: 5 hits, 8 misses*** *— hit ratio* $5/13 = 38.5\%$*, the best of the four implementable policies on* this *string. Do not over-generalise, though: LFU's weakness is stale popularity — a block referenced heavily long ago keeps a high count and refuses to leave — so on a different string LRU wins.*

Example 9.6.4 (Trace 4 — Random (One Particular Run))**.**

Reference	*7*	*0*	*1*	*2*	*0*	*3*	*0*	*4*	*2*	*3*	*0*	*3*	*2*
Slot 0	*7*	*7*	*7*	*7*	***0***	***3***	*3*	***4***	*4*	*4*	***0***	*0*	*0*
Slot 1	–	*0*	*0*	***2***	*2*	*2*	*2*	*2*	*2*	*2*	*2*	*2*	*2*
Slot 2	–	–	*1*	*1*	*1*	*1*	***0***	*0*	*0*	***3***	*3*	*3*	*3*
Hit/Miss	*M*	*M*	*M*	*M*	*M*	*M*	*M*	*M*	*H*	*M*	*M*	*H*	*H*

Result of this run: 3 hits, 10 misses *— hit ratio* 23.1%*. But this number is* not reproducible*: a different sequence of random victims gives a different count; only the* expected *hit ratio over many runs is a property of the policy. So why use it? It needs* zero bookkeeping hardware *— no counters, no recency list, no age bits — and simulation studies find it only slightly inferior to the usage-based algorithms [2, §5.2, PDF p. 203].*

Example 9.6.5 (Trace 5 — OPT (the Unattainable Benchmark))**.** *OPT evicts the line whose next use is furthest in the future:*

Reference	*7*	*0*	*1*	*2*	*0*	*3*	*0*	*4*	*2*	*3*	*0*	*3*	*2*
Slot 0	*7*	*7*	*7*	***2***	*2*	*2*	*2*	*2*	*2*	*2*	*2*	*2*	*2*
Slot 1	–	*0*	*0*	*0*	*0*	*0*	*0*	***4***	*4*	*4*	***0***	*0*	*0*
Slot 2	–	–	*1*	*1*	*1*	***3***	*3*	*3*	*3*	*3*	*3*	*3*	*3*
Hit/Miss	*M*	*M*	*M*	*M*	*H*	*M*	*H*	*M*	*H*	*H*	*M*	*H*	*H*

How the decisions are made: at reference 8 the residents are $\{2, 0, 3\}$*; their next uses are block 2 at step 9, block 3 at step 10, block 0 at step 11.* Block 0 is used furthest away, so block 0 is evicted *— the opposite of what LFU chose.* ***Result: 6 hits, 7 misses*** *— hit ratio* $6/13 = 46.2\%$*. (Standard general treatment — not from Stallings, which does not cover OPT. Use it as a yardstick, never as a design.)*

9.6.3 The Five Policies, Same Input, Different Outcomes

Table 4 puts the five runs side by side. Identical input, identical cache, and a 2× spread in hit ratio — the policy is not a detail. OPT's 46.2% is the ceiling: no implementable policy on this string can do better, so LFU's 38.5% leaves at most 7.7 percentage points on the table. One string proves nothing about policies in general — it proves that policies *diverge* — which is why real designs are chosen by simulation over real traces, not by argument.

Policy	Hits	Misses	Hit ratio	What it tracks
OPT (benchmark)	6	7	46.2%	the future (unimplementable)
LFU	5	8	38.5%	a reference counter per line
LRU	4	9	30.8%	recency order of the lines
FIFO	3	10	23.1%	load order only
Random (this run)	3	10	23.1%	nothing

Table 4: The five replacement policies run on the same 13-reference string in the same 3-line fully associative cache.

9.6.4 Belady's Anomaly: More Cache, More Misses

The most counter-intuitive fact in the chapter is that, under *FIFO*, *increasing* the number of cache lines can *increase* the number of misses on the same reference string.

Definition (Belady's Anomaly — standard general treatment, no page cite). *Under FIFO, increasing the number of cache lines can increase the number of misses on the same reference string. (Standard general treatment: Belady's anomaly appears nowhere in this course's Stallings text, so no page is cited for it.)*

Example 9.6.6 (Belady's Anomaly, Traced at 3 Lines and 4 Lines)**.** *Run the reference string* $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$ *under FIFO, first with 3 lines, then with 4.*

FIFO with 3 lines:

Reference	*1*	*2*	*3*	*4*	*1*	*2*	*5*	*1*	*2*	*3*	*4*	*5*
Slot 0	*1*	*1*	*1*	***4***	*4*	*4*	***5***	*5*	*5*	*5*	*5*	*5*
Slot 1	–	*2*	*2*	*2*	***1***	*1*	*1*	*1*	*1*	***3***	*3*	*3*
Slot 2	–	–	*3*	*3*	*3*	***2***	*2*	*2*	*2*	*2*	***4***	*4*
Hit/Miss	*M*	*M*	*M*	*M*	*M*	*M*	*M*	*H*	*H*	*M*	*M*	*H*

3 lines ⇒ ***9 misses, 3 hits****.*

FIFO with 4 lines:

Reference	*1*	*2*	*3*	*4*	*1*	*2*	*5*	*1*	*2*	*3*	*4*	*5*
Slot 0	*1*	*1*	*1*	*1*	*1*	*1*	***5***	*5*	*5*	*5*	***4***	*4*
Slot 1	–	*2*	*2*	*2*	*2*	*2*	*2*	***1***	*1*	*1*	*1*	***5***
Slot 2	–	–	*3*	*3*	*3*	*3*	*3*	*3*	***2***	*2*	*2*	*2*
Slot 3	–	–	–	*4*	*4*	*4*	*4*	*4*	*4*	***3***	*3*	*3*
Hit/Miss	*M*	*M*	*M*	*M*	*H*	*H*	*M*	*M*	*M*	*M*	*M*	*M*

The result: 3 lines gives 9 misses, but 4 lines gives ***10 misses****. A* bigger *cache performed* worse *on the identical reference string.*

The reason LRU and OPT are immune is that they are stack algorithms*: the set of blocks held with n lines is always a subset of the set held with $n + 1$ lines, so extra capacity can never remove a block that would otherwise have hit. FIFO has no such property — adding a line changes the eviction order itself, not just its depth.*

9.6.5 Implementing These Policies in Hardware

Each policy is only as good as the hardware it needs, and the cost rises with how much history the policy remembers. Random remembers nothing; exact LRU remembers everything:

- **LRU, two-way set associative** — one **USE bit** per set. Referencing way 0 sets the bit to 0, referencing way 1 sets it to 1; the victim is the way the bit does *not* point at. One bit per set, total [2, §5.2, PDF p. 203].
- **LRU, fully associative** — maintain an **index list** of all L lines in recency order. A reference moves that line to the front; the victim is the tail. Correct, but the list must be reordered on *every* access, which is why exact LRU is rare beyond small associativities [2, §5.2, PDF p. 203].
- **FIFO** — a **circular buffer** (round-robin pointer) per set. Advance the pointer on each replacement; no update at all on a hit. $\lceil \log_2 k \rceil$ bits per set.
- **LFU** — a **reference counter** per line, incremented on every access to that line; the victim is the minimum. Costs a counter and a comparison tree per set, and needs periodic ageing to stop stale counts dominating.
- **Random** — a free-running counter or LFSR shared by the whole cache. **No per-line state whatsoever**, which is exactly its appeal.

Example 9.6.7 (Why Is 2-Way LRU So Cheap?)**.** *Exact LRU over 64 fully associative lines needs a 64-entry ordered list; exact LRU over a 2-way set needs* one bit*. Where did the cost go? Recency order over n items is a* permutation*: there are $n!$ of them, needing $\lceil \log_2 n! \rceil$ bits to encode. For $n = 2$ that is $\lceil \log_2 2 \rceil = 1$ bit; for $n = 64$ it is roughly* 296 bits per set*. So associativity does not merely cost comparators (Section 9.5) — it costs* replacement bookkeeping too*, and that cost grows far faster than linearly. This is why real 8- and 16-way caches use* pseudo-*LRU (a small tree of bits that approximates recency) rather than exact LRU.*

9.7 Cache Performance

Section 9.2 introduced the two-level model qualitatively; this section puts precise numbers on the whole chapter. Anchor reading for AMAT is Patterson & Hennessy, *Computer Organization and Design*, ARM ed. 5e, Ch. 5 §5.3 (p. 397).

9.7.1 Hit Ratio, Miss Ratio, and AMAT

For n memory references of which n_h hit in the cache:

$$H = \frac{n_h}{n}, \qquad \text{miss ratio} = 1 - H,$$

$$\text{AMAT} \;=\; T_{\text{hit}} \;+\; (1 - H) \times T_{\text{miss penalty}}.$$

AMAT is the **average memory access time** seen by the CPU [1, §5.3, p. 397]. It reconciles cleanly with the two-level form of Section 9.2: that section wrote $T_{\text{avg}} = HT_1 + (1 - H)(T_1 + T_2)$, which expands to $T_1 + (1 - H)T_2$ — identical to AMAT with $T_{\text{hit}} = T_1$ and miss penalty $= T_2$. The trap to avoid: a miss penalty is the *extra* time *beyond* the hit time, because the hit time is paid on every access, hit or miss. Adding it twice is the single most common arithmetic error here.

Example 9.7.1 (AMAT from a Reference Count)**.** *A program makes 1000 memory references; 940 are found in the cache. Cache access time is 2 ns and a miss costs a further 80 ns to fetch the block from main memory. Find the hit ratio, miss ratio, and AMAT.*

- *Hit ratio* $H = 940/1000 = 0.94$*; miss ratio* $= 1 - 0.94 = 0.06$.
- $AMAT = 2 + 0.06 \times 80 = 2 + 4.8 = 6.8$ *ns.*
- *Cross-check with the two-level form:* $0.94 \times 2 + 0.06 \times (2 + 80) = 1.88 + 4.92 = 6.8$ *ns.*

Sanity test on any AMAT answer: it must lie strictly between the hit time (2 ns) and the full miss time (82 ns). 6.8 *does.*

9.7.2 Two Cache Levels: Two Access Models

With an L1 cache (time t_1, hit ratio h_1), an L2 cache (time t_2, *local* hit ratio h_2 measured over L1 misses only), and main memory (time t_m), two access models decide the arithmetic:

- **Hierarchical (sequential) access** — each level is searched only after the one above it misses, so its time *adds*:

$$T = h_1 t_1 + (1 - h_1) h_2 (t_1 + t_2) + (1 - h_1)(1 - h_2)(t_1 + t_2 + t_m).$$

- **Simultaneous access** — levels are probed in parallel, so only the level that answers contributes:

$$T = h_1 t_1 + (1 - h_1) h_2 \, t_2 + (1 - h_1)(1 - h_2) \, t_m.$$

Always read the question for which model is intended. Unless a problem says otherwise, assume hierarchical — it is what real multi-level caches do [2, §5.2 Number of Caches, PDF pp. 205–208].

Example 9.7.2 (Effective Access Time, Hierarchical)**.** *Let* $t_1 = 1$ *ns with* $h_1 = 0.95$*;* $t_2 = 10$ *ns with local* $h_2 = 0.90$*; main memory* $t_m = 100$ *ns. Find the effective access time.*

- *L1 hit:* $0.95 \times 1 = 0.95$ *ns.*
- *L1 miss, L2 hit:* $(1 - 0.95) \times 0.90 \times (1 + 10) = 0.045 \times 11 = 0.495$ *ns.*
- *Both miss:* $(1 - 0.95) \times (1 - 0.90) \times (1 + 10 + 100) = 0.005 \times 111 = 0.555$ *ns.*
- ***Total*** $T = 0.95 + 0.495 + 0.555 = 2.0$ ***ns.***
- *Weight check:* $0.95 + 0.045 + 0.005 = 1.000$ *— the three probabilities must sum to 1, or a case has been missed.*

Compared with 100 ns raw DRAM, the two-level hierarchy delivers a $50\times$ *effective speed-up.*

Example 9.7.3 (Same Numbers, Simultaneous Model — and a Warning)**.** *Recompute with parallel probing:* $T = 0.95(1) + 0.045(10) + 0.005(100) = 0.95 + 0.45 + 0.50 = 1.9$ *ns. Only* 0.1 *ns apart here — but the gap widens fast as* h_1 *falls, because every miss then pays* t_1 *twice as often under the hierarchical model.*

The other classic trap is local vs. global *hit ratio. Here* $h_2 = 0.90$ *is* local *— 90% of the references that* reach *L2. The* global *L2 hit ratio is* $(1 - h_1)h_2 = 0.05 \times 0.90 = 0.045$*, i.e. only 4.5% of all references. If a problem states an L2 hit ratio without saying which, the phrase "of the accesses that miss in L1" marks it local; "of all memory references" marks it global. Using the wrong one is a full-marks error, not a rounding one.*

Example 9.7.4 (Where Should the Money Go?)**.** *Starting from $t_1 = 1$ ns, $h_1 = 0.95$, $t_m = 100$ ns, and no L2 (so AMAT $= 1 + 0.05 \times 100 = 6$ ns), you may spend your budget on exactly one of: (a) halving t_1 to 0.5 ns, or (b) raising h_1 to 0.975. Which wins?*

- *(a)* $0.5 + 0.05 \times 100 = 5.5$ *ns — a* 0.5 *ns saving.*
- *(b)* $1 + 0.025 \times 100 = 3.5$ *ns — a* 2.5 *ns saving,* five times better.

The lesson: when the miss penalty is large, the miss rate *dominates AMAT, and hit time is a second-order term. Halving the miss rate is worth more than halving the hit time — which is exactly why Section 9.5 spent so long on mapping and Section 9.6 on replacement: both are miss-rate levers.*

9.7.3 Two Design Knobs Not Yet Priced: Write Policy and Line Size

Two of the seven knobs of Section 9.4 are not part of the mapping/replacement story but change the two terms AMAT is built from, so they close this section.

Write policy decides what happens when the CPU *writes*. **Write through** updates cache *and* main memory on every write: memory is always consistent, but write traffic is high. **Write back** updates only the cache and sets the line's **dirty bit**; main memory is updated only when a dirty line is evicted: far less traffic, but memory is temporarily stale — a real problem for I/O modules and for other processors' caches [2, §5.2 Write Policy, PDF pp. 203–205].

Line size has a maximum, not a monotone benefit: larger blocks exploit spatial locality, but past some point they fetch data that is never used and evict data that still is, so the hit ratio turns back down [2, §5.2 Line Size, PDF p. 205].

9.8 Synthesis: The Threads Meet

Every cache decision buys hit ratio and pays in hardware or time — the same "no free lunch, only a choice of who pays" pattern that Weeks 5–8 established for buses, control units, and I/O.

Hierarchy vs. one memory: locality lets a 2 KB SRAM stand in for 64 KB of DRAM most of the time — recall from Example 9.2.1 that $H = 0.95$ turned 100 ns into 6 ns.

Mapping (Section 9.5): direct mapping costs 1 comparator and 320 tag bits but suffers 0% hit ratio on a two-block conflict loop (Example 9.5.4); fully associative removes every conflict miss and costs 64 comparators and 704 tag bits. Set associativity is the dial between them.

Replacement (Section 9.6): on one identical 13-reference string, FIFO scored 3 hits, LRU 4, LFU 5, and the unattainable OPT 6 — and FIFO can be made *worse* by a bigger cache (Belady's anomaly, Example 9.6.6).

Performance (Section 9.7): AMAT is dominated by miss rate when the miss penalty is large (Example 9.7.4) — which is why mapping and replacement mattered.

Week 9 made a small fast memory stand in for a large slow one, using blocks, tags, mapping, and replacement. Week 10 applies the identical machinery between *main memory and disk* — and calls it **virtual memory**: cache line becomes page, tag lookup becomes page table, miss becomes page fault, and the TLB is itself a small associative cache of translations. Replacement returns too, and this time Belady's anomaly is not a curiosity but a documented hazard of FIFO page replacement.

9.8.1 Summary

Hierarchy: capacity, cost/bit, and access time cannot all improve together; **temporal and spatial locality** make a small fast level useful. Two-level model $T_{\text{avg}} = T_1 + (1 - H)T_2$.

Main memory: SRAM (flip-flop, no refresh, cache) vs. DRAM (capacitor, refresh, main memory); the ROM family runs from mask ROM (unchangeable) through PROM, EPROM (UV erase), EEPROM (byte erase) to flash (block erase).

Mapping — one formula, three settings: tag | index | word, with index = $\log_2 L$ (direct), $\log_2(L/k)$ (k-way), or 0 bits (fully associative), and the tag absorbing every bit the index gives up. For System A: 5 | 6 | 5, 7 | 4 | 5, 11 | 5; address `0x3F8C` maps to line 60/tag 7, set 12/tag 31, and tag 508.

Cost: comparators = 1, k, or L — doubling associativity doubles the comparators but adds only one tag bit per line.

Replacement: needed only where placement offers a choice (*never* under direct mapping). LRU (USE bit / index list), FIFO (circular buffer), LFU (per-line counter), Random (no state), and OPT as a benchmark. **Belady's anomaly**: FIFO can miss *more* with a larger cache; LRU and OPT, being stack algorithms, cannot.

Performance: AMAT = $T_{\text{hit}} + (1-H) \times$ miss penalty; multi-level effective access time depends on whether access is hierarchical or simultaneous, and on whether h_2 is local or global.

Exercises

1. A system has 1 MB main memory, a 32 KB cache, and 64-byte blocks. Derive, showing every step as in Example 9.5.2, the tag/index/offset field widths for (a) direct mapping and (b) 8-way set-associative mapping, and verify each re-sums to the address width.

2. For the direct-mapped cache of Exercise 1, a loop alternately references two addresses whose block numbers are 100 and 612. Using the method of Example 9.5.4, show whether the two blocks collide, and state the hit ratio if the loop runs.

3. Trace the reference string $2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2$ under FIFO in a 3-line fully associative cache (following Example 9.6.1), and report the hit ratio.

4. Using the AMAT form of Example 9.7.1, compute AMAT for a cache with hit ratio 0.92, hit time 1 ns, and miss penalty 50 ns, and state the two bounds any correct answer must lie between.

5. Following Example 9.7.2, recompute the effective access time for $t_1 = 2$ ns, $h_1 = 0.90$, $t_2 = 20$ ns, local $h_2 = 0.80$, and $t_m = 200$ ns under the hierarchical model, and give the global L2 hit ratio (as in Example 9.7.3).

6. Explain, in one or two sentences each, (a) why OPT is unimplementable, (b) what Belady's anomaly is and which policies are immune to it, and (c) why exact LRU is impractical for a 64-line fully associative cache.

Solutions

1. Main memory = 1 MB = 2^{20} bytes, so $N = 20$. Block = 64 = 2^6 bytes, so $b = 6$. Lines L = 32 KB/64 B = $2^{15}/2^6 = 2^9 = 512$. (a) Direct: index = $\log_2 512 = 9$ bits, tag = $20 - 9 - 6 = 5$ bits; check $5 + 9 + 6 = 20$. (b) 8-way: $S = 512/8 = 64 = 2^6$, set field = 6 bits, tag = $20 - 6 - 6 = 8$ bits; check $8 + 6 + 6 = 20$.

2. Block 100: $100 \bmod 512 = 100$; block 612: $612 \bmod 512 = 612 - 512 = 100$. Both map to line 100 with tags $\lfloor 100/512 \rfloor = 0$ and $\lfloor 612/512 \rfloor = 1$, so every reference finds the other block's tag — hit ratio 0%, exactly the conflict miss of Example 9.5.4.

3. FIFO with 3 lines, round-robin slots 0,1,2. Fill: 2 →slot0, 3 →slot1, (2 hits), 1 →slot2, 5 evicts slot0(2), 2 evicts slot1(3), 4 evicts slot2(1), 5 hits, 3 evicts slot0(5), 2 hits, 5 evicts slot1(2), 2 evicts slot2(4). Hits at references 3, 8, and 10 — 3 hits, 9 misses, hit ratio $3/12 = 25\%$.

4. AMAT $= 1 + (1 - 0.92) \times 50 = 1 + 0.08 \times 50 = 1 + 4 = 5$ ns. Any correct AMAT must lie strictly between the hit time (1 ns) and the full miss time ($1 + 50 = 51$ ns); 5 ns does.

5. L1 hit: $0.90 \times 2 = 1.8$. L1 miss, L2 hit: $(0.10)(0.80)(2 + 20) = 0.08 \times 22 = 1.76$. Both miss: $(0.10)(0.20)(2 + 20 + 200) = 0.02 \times 222 = 4.44$. Total $T = 1.8 + 1.76 + 4.44 = 8.0$ ns. Weights: $0.90 + 0.08 + 0.02 = 1.00$. Global L2 hit ratio $= (1 - h_1)h_2 = 0.10 \times 0.80 = 0.08$ (8%).

6. (a) OPT evicts the block whose next use is furthest in the future, which requires knowing the future reference stream — no real cache has it, so OPT is a benchmark, not a policy. (b) Belady's anomaly is FIFO's property of missing *more* when the cache grows; LRU and OPT are stack algorithms and so are immune. (c) Exact LRU over 64 lines needs an ordered 64-entry list reordered on every access (~ 296 bits of permutation state), which is far too much for cache speed; 2-way LRU needs one bit, which is why real wide caches use pseudo-LRU.

References

[1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.

[2] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.